

S-Dummy REST API Testing Project

Test Plan (IEEE Style)

Version: 1.0

Prepared By: Mahbub Surov

Project Type: Manual REST API Testing

Tool Used: Postman

API: DummyJSON REST API

Table of Contents

1. Project Overview
2. Test Objectives
3. Scope
4. Modules Covered
5. Test Environment
6. Test Strategy
7. Test Approach
8. Entry Criteria
9. Exit Criteria
10. Test Deliverables
11. Risks and Assumptions

1. Project Overview

The S-Dummy REST API Testing Project is a manual API testing project created using Postman to verify the functionality of the DummyJSON REST API.

The project focuses on validating authentication, CRUD operations, search functionality and error handling by executing both positive and negative test scenarios.

The objective is to demonstrate practical API testing skills through professional QA documentation and organized Postman collections.

2. Test Objectives

- Verify Authentication APIs.
- Validate CRUD operations.
- Test Product APIs.

- Test Cart APIs.
- Test User APIs.
- Verify Search functionality.
- Validate error handling using Negative Testing.
- Ensure APIs return correct HTTP status codes.
- Verify response structure and basic performance.

3. Scope

In Scope

Module	Coverage
Authentication	Login, Profile
Products	CRUD, Search, Categories, Sorting
Cart	CRUD
Users	Get, Search, Update, Delete
Negative Testing	Invalid Requests

Out of Scope

- Database Testing
- UI Testing
- Security Penetration Testing
- Load Testing
- Performance Benchmarking
- Mobile Testing

4. Modules Covered

Authentication

- Login user
- Get current profile
- Bearer Token validation

Products

- Get products
- Get single product
- Add product
- Update product
- Delete product
- Search product
- Product categories
- Sorting

Cart

- Get cart
- Add cart
- Update cart

- Delete cart

Users

- Get users
- Search users
- Update user
- Delete user

Negative Testing

- Wrong login
- Invalid product
- Invalid user
- Invalid cart
- Wrong endpoint
- Wrong HTTP method
- Blank search

5. Test Environment

Item	Details
Operating System	Windows
API Client	Postman
API	DummyJSON
Authentication	Bearer Token
Collection	S-Dummy Collection
Environment	S-Dummy Environment

Environment Variables

Variable	Purpose
base-url	Base API URL
token	Bearer Token
product-id	Product Testing
cart-id	Cart Testing
user-id	User Testing

6. Test Strategy

The project follows a Manual Functional API Testing strategy.

Positive Testing

- Successful Login
- Get Products
- Add Product
- Update User

Negative Testing

- Wrong Password
- Invalid Product ID
- Wrong Endpoint

- Blank Search

Validation Performed

- Status Code Validation
- Response Body Validation
- Authentication Validation
- Basic Response Time Check

7. Test Approach

1. Import the Postman Collection.
2. Import the Environment.
3. Run the Login request.
4. Store the Bearer Token.
5. Execute each module individually.
6. Run Negative Test scenarios.
7. Record Test Cases and Bug Reports.

8. Entry Criteria

- Postman is installed.
- Collection is imported.
- Environment is configured.
- Internet connection is available.
- DummyJSON API is accessible.

9. Exit Criteria

- All planned APIs are executed.
- Authentication flow is verified.
- CRUD operations are completed.
- Negative Test scenarios are executed.
- Test Cases are documented.
- Bug Reports are prepared.
- Test Summary is completed.

10. Test Deliverables

Deliverable	Status
Postman Collection	Completed
Environment File	Completed
Manual Test Cases	Completed
Bug Report	Completed
Test Plan	Completed

RTM	Planned
Test Summary Report	Planned
Newman Report	Completed

11. Risks and Assumptions

Risks

- Internet connectivity may affect API execution.
- API behavior may change in future updates.
- Response time may vary depending on network conditions.

Assumptions

- DummyJSON API remains available.
- Valid authentication credentials are used.
- Environment variables are correctly configured before execution.

Approval

Item	Value
Project	S-Dummy REST API Testing
Version	1.0
Prepared By	Mahbub Sourov
Testing Type	Manual API Testing
Tool	Postman